

Projeto e Implementação de um Sistema de Arquivos Distribuído (DFS)

Objetivo Geral

Neste trabalho, os alunos deverão projetar e implementar um **Sistema de Arquivos Distribuído (Distributed File System - DFS)** capaz de operar em ambiente distribuído real ou simulado.

O sistema desenvolvido deve atender, obrigatoriamente, às seguintes propriedades fundamentais:

- **Alta Escalabilidade**
O sistema deve suportar crescimento horizontal, permitindo a adição e remoção dinâmica de nós, mantendo desempenho aceitável sob aumento de carga e volume de dados.
- **Tolerância a Falhas**
O sistema deve continuar operando corretamente mesmo na presença de falhas parciais, incluindo:
 - Falha de nós
 - Falhas de rede
 - Perda ou atraso de mensagensGarantindo **disponibilidade e integridade dos dados**.

Requisitos Técnicos

O sistema deve implementar, no mínimo, os seguintes componentes e funcionalidades:

Armazenamento Distribuído

- Arquivos devem ser distribuídos entre múltiplos nós
- Suporte a operações básicas: write, read, delete, list

Replicação de Dados

- Replicação de arquivos entre nós distintos
- Definição explícita do fator de replicação
- Estratégia de posicionamento de réplicas

Consistência de Dados

- Implementação de uma estratégia de consistência:

- Forte, eventual ou configurável
- Justificativa formal da escolha

Detecção e Tratamento de Falhas

- Mecanismos como:
 - Heartbeat
 - Timeout
 - Retry
- Estratégias de:
 - Detecção de falhas
 - Reconfiguração do cluster

Balanceamento de Carga

- Distribuição equilibrada de dados e requisições
- Estratégias como hashing consistente ou similares

Recuperação de Dados

- Re-replicação automática após falhas
- Reconstrução de dados a partir de réplicas

Interface de Acesso

- Interface para interação com o sistema:
 - API (REST/gRPC), CLI ou interface simples
- Documentação de uso

Requisitos de Escalabilidade (Obrigatórios)

O sistema deve demonstrar:

- **Adição e remoção dinâmica de nós (elasticidade)**
- **Execução de testes de carga**, variando:
 - Número de nós
 - Volume de dados
 - Taxa de requisições

- **Análise de desempenho**, incluindo:
 - Latência
 - Throughput
- **Identificação de gargalos**
- Discussão dos limites do sistema

Requisitos de Tolerância a Falhas (Obrigatórios)

O sistema deve incluir:

- **Simulação de cenários de falha**, tais como:
 - Queda de nós
 - Perda de conectividade
 - Atraso de mensagens
- **Mecanismos de recuperação automática**
- Garantias explícitas de:
 - Disponibilidade (sistema continua respondendo)
 - Integridade (dados não corrompidos)

Estrutura por Marcos (Milestones)

O desenvolvimento será incremental, dividido em cinco marcos obrigatórios.

◆ **Marco 1 — Fundamentos (Baixa Complexidade)**

Objetivo: Construir a base do sistema.

Funcionalidades:

- Arquitetura cliente-servidor simples
- Armazenamento local de arquivos em um único nó
- Interface básica (CLI ou API)

CrITÉrios de Avaliação:

- Correção funcional

- Clareza da arquitetura inicial
- Organização do código

◆ **Marco 2 — Distribuição (Complexidade Média)**

Objetivo: Introduzir distribuição de dados.

Funcionalidades:

- Múltiplos nós de armazenamento
- Particionamento de arquivos (sharding)
- Mecanismo de roteamento de requisições

CrITÉRIOS de Avaliação:

- Distribuição correta dos dados
- Balanceamento inicial
- Comunicação entre nós

◆ **Marco 3 — Replicação e Consistência (Complexidade Média-Alta)**

Objetivo: Garantir redundância e definir consistência.

Funcionalidades:

- Replicação de arquivos
- Definição de política de consistência
- Controle de versões ou timestamps

CrITÉRIOS de Avaliação:

- Coerência dos dados replicados
- Implementação correta da política escolhida
- Justificativa técnica

◆ **Marco 4 — Tolerância a Falhas (Alta Complexidade)**

Objetivo: Tornar o sistema resiliente.

Funcionalidades:

- Detecção de falhas (heartbeat, timeout)
- Recuperação automática
- Eleição de líder (se necessário)

- Re-replicação de dados

Critérios de Avaliação:

- Capacidade de recuperação
- Robustez sob falhas simuladas
- Continuidade do serviço

◆ Marco 5 — Escalabilidade e Avaliação (Alta Complexidade)

Objetivo: Validar comportamento em escala.

Funcionalidades:

- Testes de carga automatizados
- Coleta de métricas (latência, throughput)
- Análise de desempenho
- Identificação de gargalos

Critérios de Avaliação:

- Qualidade dos experimentos
- Profundidade da análise
- Coerência entre arquitetura e resultados

Critérios de Avaliação

A nota final será composta por:

- **Correção funcional** (20%)
- **Tolerância a falhas** (20%)
- **Escalabilidade demonstrada** (20%)
- **Qualidade do código** (15%)
- **Documentação técnica** (15%)
- **Análise experimental** (10%)

Entregáveis

Para cada marco e entrega final:

- Código-fonte versionado
- Relatório técnico contendo:
 - Arquitetura
 - Decisões de projeto
 - Experimentos realizados
- Scripts de teste e simulação
- Demonstração prática (apresentação)

Nível de Profundidade Exigido

Os alunos deverão:

- Justificar decisões arquiteturais
- Comparar alternativas (ex: consistência forte vs eventual)
- Analisar trade-offs clássicos:
 - Teorema CAP
 - Latência vs consistência
 - Replicação vs custo

Diferenciais (Bônus)

Serão valorizadas implementações que incluam:

- Uso de containers (Docker)
- Orquestração (Docker Compose ou Kubernetes)
- Simulação realista de rede distribuída
- Implementação de protocolos como:
 - Raft (consenso)
 - Gossip (disseminação de estado)

- Observabilidade (logs estruturados, métricas, tracing)

Considerações Finais

Este trabalho simula desafios reais enfrentados por sistemas como Google File System (GFS), miniIO e Hadoop HDFS. Espera-se que o aluno não apenas implemente funcionalidades, mas demonstre **maturidade em engenharia de sistemas distribuídos**, com forte capacidade de análise, experimentação e justificativa técnica.